

LangGraph ReAct Architecture

Orchestrator + Worker State Graphs

Iqra Digital Library v2 | May 2026

System Overview

The Iqra Digital Library uses a three-layer LangGraph ReAct hierarchy. A central Orchestrator receives every user message and routes it to two specialist Worker agents ? SearchWorker for book discovery and CuratorWorker for reading list management. Each layer is an independent ReAct graph: `__start__ -> agent -> tools -> agent -> ... -> __end__`. The Orchestrator holds MemorySaver conversation memory per session. Workers are stateless ? each call is a fresh graph invocation.

Contents

1. State Graph ? Visual Diagram (all three agents)
2. Graph Anatomy ? Nodes, Edges & Message Types
3. Orchestrator (LibraryAgent) ? Role, Logic, Design
4. Search Worker ? Role, 4 Tools, Decision Tree, Retrieval Stack
5. Curator Worker ? Role, 4 Tools, Decision Tree, Data Schema
6. Message Flow ? End-to-end walkthroughs
7. Memory & State ? MemorySaver, `thread_id`, Checkpoints
8. Design Decisions ? Why this architecture

1. State Graph ? All Three Agents

The diagram below shows the actual LangGraph-generated state graph topology for the Orchestrator and both Worker agents. Every graph follows the same ReAct loop structure. What differs is only the tools available inside the 'tools' execution node.

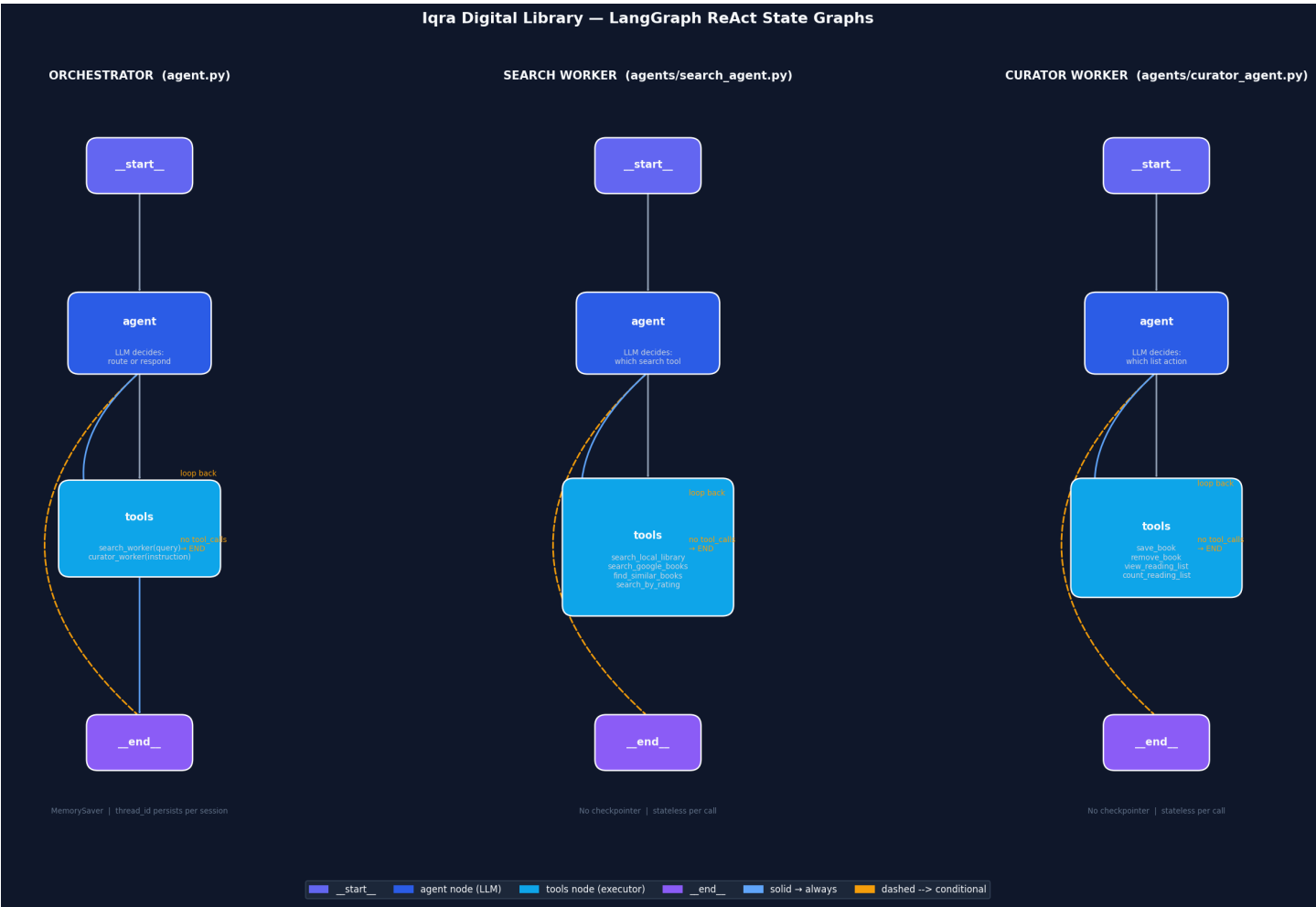


Figure 1 ? Orchestrator (left) | Search Worker (centre) | Curator Worker (right)

Node Colour Legend

Purple <code>__start__</code>	Entry point ? where the user input arrives
Blue <code>agent</code>	LLM node ? reads message history, decides next action
Teal <code>tools</code>	Executor node ? runs whichever tool the LLM chose
Violet <code>__end__</code>	Exit point ? emits the final response to the caller
Solid arrow	Always taken (tools -> agent loops unconditionally)
Dashed arrow	Conditional (agent -> <code>__end__</code> only when no tool_calls)

2. Graph Anatomy ? Nodes, Edges & Message Types

2.1 The Four Nodes (present in every graph)

Node	Type	What it does	When it runs
__start__	Entry	Injects the incoming HumanMessage into graph state	Once per invoke() call, before anything else.
agent	LLM call	Passes full message history to the LLM. Returns AI	Every iteration: first to decide, then again after each tool message
tools	Executor	Executes every tool_call from the last AIMessage.	Only when agent node outputs tool_calls. Skipped when LLM
__end__	Exit	Terminates the graph. Returns final state (all mess	When agent produces an AIMessage with NO tool_calls.

2.2 The Four Edges

Edge	Type	Condition / Reason
__start__ -> agent	Static	Always. Graph always starts by asking the LLM what to do next.
agent -> tools	Conditional	Taken when LLM last message contains tool_calls (it wants to run a tool).
agent -> __end__	Conditional	Taken when LLM last message has NO tool_calls (producing final answer).
tools -> agent	Static	Always. After every tool run, control returns to LLM to decide the next step.

2.3 Message Types in State (MessagesState list)

The graph state is a single list of BaseMessage objects. Every node reads the full list and appends its output. The LLM always receives the complete list as conversation context.

Message Type	Created by	Contains
HumanMessage	User / invoke()	Raw user text. Added before __start__ on every chat() call.
AIMessage (tool_calls)	agent LLM	Decision to call tools. Includes tool name + arguments as structured JSON.
ToolMessage	tools node	String result from a tool function. One ToolMessage per tool_call in the list.
AIMessage (final)	agent LLM	Natural-language reply with no tool_calls. Triggers transition to __end__.
SystemMessage (prompt=)	Graph init	System prompt injected at position 0 of every agent call. NOT stored in checkpoint.

3. Orchestrator ? LibraryAgent (agent.py)

3.1 Role

The Orchestrator is the single entry point for all user messages. app.py calls agent_mgr.chat(message, thread_id) ? that method invokes this graph. The Orchestrator does NOT answer directly. Its job is to classify the user's intent and route to the correct specialist worker, then synthesise the worker result into a coherent reply.

3.2 Key Properties

File: agent.py

Class:

3.3 Decision Logic

The Orchestrator LLM routes with these rules (from the system prompt):

- Search / discovery / recommendations -> call search_worker(query)
- Reading list: save / remove / view / count -> call curator_worker(instruction)
- Compound ('find X and save it') -> call search_worker, then curator_worker with result
- Conversational (greeting, clarification) -> respond directly, no worker needed

3.4 System Prompt

You are Iqra, a friendly digital library assistant.

Two specialist workers you MUST delegate to:

search_worker -> book discovery, recommendations, similar books, Google Books
curator_worker -> save / remove / view / count reading list

Rules:

- Any book question -> search_worker
- Any reading list op -> curator_worker
- Compound request -> search_worker THEN curator_worker
- Never answer book questions from memory alone. Always delegate.

3.5 Why Use an Orchestrator?

- Single responsibility per agent: each worker has a focused prompt optimised for its task
- Fewer tools = fewer mistakes: with 2 tools the LLM almost never routes incorrectly
- Compound requests handled naturally: 'find X and save it' calls both workers in sequence
- Easier debugging: isolate whether failure is at routing, search, or curation
- Scalability: add a new worker (AnalyticsWorker, ExportWorker) without touching others

3.6 Build Sequence

- 1. llm_provider.get_langchain_llm() -> LangChain BaseChatModel

- 2. SearchWorker.build() -> builds inner ReAct graph with 4 search tools
- 3. CuratorWorker.build() -> builds inner ReAct graph with 4 list tools
- 4. create_react_agent(llm, [search_worker_tool, curator_worker_tool], checkpoint=MemorySaver(), prompt=...)
- 5. Stored as agent_mgr._agent ? ready for chat()

4. Search Worker (agents/search_agent.py)

4.1 Role

SearchWorker is a focused ReAct agent whose only job is to find books. It is invoked by the Orchestrator via the `search_worker @tool` wrapper. It runs its own complete ReAct loop ? decides which search tool to call, calls it, reads the result, and returns a formatted book list string back to the Orchestrator. It has no memory: every call starts fresh.

4.2 Key Properties

File: agents/search_agent.py

Class:

4.3 Tools

Tool	Triggered when...	Retrieval method
search_local_library	General book query	HybridRetriever: BM25 + FAISS + KG 1-hop expansion. Top 6 results.
search_google_books	Local results insufficient / user asks for more	Live Google Books API. Up to 6 external results with covers + links.
find_similar_books	'Books like X' / 'Similar to X'	BookRecommender FAISS cosine similarity on reference title embedding.
search_by_rating	'Top rated' / 'Highly rated X'	HybridRetriever k=20 -> filter average_rating >= min_rating (default 4.0) -> top 6

4.4 Decision Tree

- User: 'find me a fantasy book' -> search_local_library('fantasy')
- User: 'books like Dune' -> find_similar_books('Dune')
- User: 'top rated mystery novels' -> search_by_rating('mystery novels', 4.0)
- User: 'any new sci-fi releases?' -> search_google_books('new sci-fi 2025')
- Local returns nothing -> agent may chain to search_google_books

4.5 Hybrid Retrieval Stack (what happens inside search_local_library)

- Stage 1 ? FAISS dense search: query embedded with SentenceTransformer -> cosine similarity over book_index.faiss -> top k*4 candidates.
- Stage 2 ? Knowledge Graph expansion: each candidate expands 1 hop through the NetworkX KG. Edges: SAME_AUTHOR (w=1.00), SAME_CATEGORY (w=0.75), SEMANTIC_SIM cosine>=0.82. Expanded neighbours get graph_score = edge_weight.
- Stage 3 ? Score fusion: min-max normalise both streams, fuse: final_score = 0.65 * faiss_score + 0.35 * graph_score. Re-rank. Return top k.

4.6 Why No Memory?

Workers are tools from the Orchestrator's perspective ? semantically equivalent to search_local_library. Tools don't hold memory. The Orchestrator already has full conversation history in MemorySaver. If workers also held memory, state would be duplicated at two layers. The

5. Curator Worker (agents/curator_agent.py)

5.1 Role

CuratorWorker is a focused ReAct agent whose only job is to manage the user's personal reading list. Invoked by the Orchestrator via the curator_worker @tool. Runs its own complete ReAct loop and is stateless per call. Reads/writes data/reading_list.json via the ReadingList class.

5.2 Key Properties

File: agents/curator_agent.py

Class:

5.3 Tools

Tool	Triggered when...	What it does
save_book	'Save this book' / 'Add X to my list'	ReadingList.add(book_dict). Deduplicates by title. Writes JSON. Returns confirm
remove_book	'Remove X' / 'Delete X from my list'	ReadingList.remove(title). Exact title match. Returns confirmation or 'not found'.
view_reading_list	'Show my list' / 'What have I saved?'	ReadingList.get_all(). Returns numbered list: title, author, rating, date saved.
count_reading_list	'How many books?' / 'How big is my list'	ReadingList.count. Returns 'You have N books in your reading list.'

5.4 Decision Tree

- User: 'save Harry Potter' -> save_book(title='Harry Potter')
- User: 'save Dune by Frank Herbert 4.8' -> save_book('Dune', 'Frank Herbert', 4.8)
- User: 'remove Dune from my list' -> remove_book('Dune')
- User: 'show my reading list' -> view_reading_list()
- User: 'how many books have I saved?' -> count_reading_list()

5.5 Reading List Data Schema (data/reading_list.json)

```
[
  {
    "title": "Dune",
    "authors": "Frank Herbert",
    "average_rating": 4.8,
    "source": "AI Agent",
    "info_link": "#",
    "added_at": "2025-05-28T09:13:00.000000"
  }
]
```

5.6 Why a Separate Curator?

- Focused prompt: curator LLM instructed only on list operations ? never confuses 'save' with 'search'
- Separation of concerns: search and list management have different failure modes and edge cases
- Testability: CuratorWorker can be unit-tested with a mock ReadingList, no retrieval stack needed
- Extensibility: add export-to-PDF, share-list, or rate-a-book as new tools here without touching search

6. Message Flow ? End-to-End Walkthroughs

6.1 Simple Search Request

User: "Find me a mystery book set in Cairo"

```
invoke({'messages': [HumanMessage('Find me a mystery book set in Cairo')]}))

[ORCHESTRATOR ? agent node]
  In: [SystemMsg, HumanMsg]
  Out: AIMessage(tool_calls=[search_worker('mystery book set in Cairo')])

[ORCHESTRATOR ? tools node]
  Executes: search_worker('mystery book set in Cairo')
  |
  +-- [SEARCH WORKER ? agent node]
  |   In: [SearchSystemMsg, HumanMsg]
  |   Out: AIMessage(tool_calls=[search_local_library('mystery Cairo')])
  |
  +-- [SEARCH WORKER ? tools node]
  |   Runs: search_local_library('mystery Cairo')
  |   Returns: ToolMessage('1. Death on the Nile... 2. ...')
  |
  +-- [SEARCH WORKER ? agent node]
  |   In: [SystemMsg, Human, AI+tools, ToolMsg]
  |   Out: AIMessage('Here are great mysteries set in Cairo...') <- no tool_calls
  |   -> __end__ (SearchWorker returns this string)
  |
  Returns: ToolMessage('Here are great mysteries set in Cairo...')

[ORCHESTRATOR ? agent node]
  In: [SysMsg, Human, AI+tools, ToolMsg(worker result)]
  Out: AIMessage('Here are some brilliant mystery books set Cairo!') <- no tool_calls
  -> __end__

User sees: 'Here are some brilliant mystery books set in Cairo!'
```

6.2 Compound Request (Find + Save)

User: "Find me a fantasy book and save the first one to my list"

```
[ORCHESTRATOR ? agent node]
  Out: AIMessage(tool_calls=[search_worker('fantasy book')])

[ORCHESTRATOR ? tools node]
  Runs: search_worker -> '1. The Name of the Wind by Patrick Rothfuss (4.5)...'

[ORCHESTRATOR ? agent node]
  Sees search result. User also asked to save first book.
  Out: AIMessage(tool_calls=[curator_worker('Save The Name of the Wind by Patrick Rothfuss')])

[ORCHESTRATOR ? tools node]
  Runs: curator_worker -> save_book('The Name of the Wind', 'Patrick Rothfuss')
  Returns: 'The Name of the Wind added to your reading list.'

[ORCHESTRATOR ? agent node]
```

```
Out: AIMessage('Found great fantasy books! I also saved the first one...') <- final
-> __end__
```

7. Memory & State ? MemorySaver, thread_id, Checkpoints

7.1 How MemorySaver Works

MemorySaver is a LangGraph checkpointer. After every node execution, it serialises the current graph state (the messages list) and stores it in RAM keyed by thread_id + checkpoint_id. On the next invoke() call with the same thread_id, LangGraph restores the last checkpoint and prepends it ? giving the agent full conversation history without the caller needing to manage it.

- Thread ID: set per user session in app.py. Each Gradio session generates a unique UUID.
- Scope: only the Orchestrator has a checkpointer. Workers are stateless (no thread_id).
- Persistence: MemorySaver is in-process RAM only. App restart clears all sessions.
- Production upgrade: replace with SqliteSaver or PostgresSaver for cross-restart durability.

7.2 State Schema

```
# LangGraph MessagesState ? used by all three graphs
class MessagesState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

# add_messages reducer: new messages are appended, never overwritten.
# The agent node reads messages[-N:] as LLM context (full history).
# The tools node appends one ToolMessage per tool_call.
```

7.3 Multi-Turn Conversation (same thread_id)

```
Turn 1: state = [System, Human('find mystery')]
        agent -> search_worker -> ToolMsg -> agent
        Final: AIMessage('Here are mysteries...') -- checkpoint saved

Turn 2: state = [...Turn1..., Human('save the first one')]
        agent -> curator_worker('Save Death on the Nile') -> ToolMsg -> agent
        Final: AIMessage('Saved!') -- checkpoint saved
        'the first one' refers to Turn 1 result -- possible via MemorySaver

Turn 3: state = [...Turn1...Turn2..., Human('show my reading list')]
        agent -> curator_worker('view reading list') -> ToolMsg -> agent
        Final: AIMessage('Your list has 1 book: Death on the Nile...')
```

7.4 Resetting State

agent_mgr.reset() sets self._agent = None and calls reset() on both workers. Called before re-configuring the LLM. All MemorySaver checkpoints are discarded. New Gradio sessions start with a fresh thread_id, naturally giving them no history.

8. Design Decisions

8.1 Why LangGraph over LangChain Chains?

Dimension	LangChain (chains)	LangGraph (graphs)
Control flow	Linear ? A then B then C	Cyclic ? can loop, branch, retry
Agentic loops	Requires manual while-loops	Native: agent->tools->agent is the graph
State	Passed explicitly between steps	Shared TypedDict, every node reads/writes
Memory	Manual injection per call	Checkpointers built in (MemorySaver, SQLite)
Debugging	Chain is opaque	Every node transition traceable in LangSmith
Multi-agent	Not native	Supervisor: workers as tools of orchestrator

8.2 Why Orchestrator + Workers vs. One Big Agent?

- Focus: one agent with 8 tools will sometimes pick the wrong one. Two agents with 4 each almost never do.
- Prompt quality: search prompt can say 'always call a tool' without conflicting with curation prompt.
- Failure isolation: if search breaks, curation still works. Single-agent failure cascades everywhere.
- Scalability: new worker (AnalyticsWorker) = new file + one @tool in agent.py. Zero changes elsewhere.
- Testability: each worker is independently callable via worker.run('query') with no other dependencies.

8.3 Why Stateless Workers?

- Memory at multiple layers creates inconsistency: which layer's version of Turn 1 wins?
- Workers are tools: search_worker('mystery') is semantically a tool call, not a conversation.
- Simpler debugging: worker behaviour depends only on its input, not prior invocations.
- Session isolation: two concurrent users have isolated MemorySavers via thread_id. Workers have nothing to isolate.

8.4 Why prompt= instead of state_modifier= / messages_modifier=?

LangGraph 1.x changed the create_react_agent API. In LangGraph 0.2.x the parameter was state_modifier=SystemMessage(...). In LangGraph 1.0 it was briefly messages_modifier=. From LangGraph 1.2 onwards the parameter is prompt= (accepts a plain string or SystemMessage). The prompt is injected at position 0 of every agent call. It is NOT stored in the checkpoint ? keeping state lean and allowing prompt updates without invalidating existing sessions.

8.5 Multi-Provider LLM Support

All three agents call llm_provider.get_langchain_llm() which returns a LangChain BaseChatModel (ChatAnthropic, ChatOpenAI, ChatGoogleGenerativeAI, or ChatOllama). Because create_react_agent accepts any BaseChatModel, the entire three-graph hierarchy works identically regardless of which provider is connected.

- Claude (claude-sonnet-4-6) ? best reasoning and tool accuracy. Needs ANTHROPIC_API_KEY.
- OpenAI (gpt-4o) ? strong tool-calling, widely known. Needs OPENAI_API_KEY.
- Gemini (gemini-1.5-pro) ? Google infrastructure. Needs GEMINI_API_KEY.
- Ollama (llama3.2, qwen2.5, mistral) ? fully local, free, no key. Needs Ollama server running.
- Note: only models supporting function/tool calling work with ReAct. gemma and phi3 excluded.